

# Software Requirements Specification & System Architecture Specification

**Project Title:** Network Device Inventory Manager  
**Document Version:** 2.0.0 (Unified Phase 1 & Phase 2 Specification)  
**Date:** August 5, 2026  
**Status:** Approved / Phase 1 & Phase 2 Sign-Off

## 1. Executive Summary & Purpose

The **Network Device Inventory Manager** is a specialized Web Application designed for Network Engineers and System Administrators to systematically track, monitor, and document enterprise network assets. It solves the critical problem of unmanaged infrastructure assets, outdated spreadsheet inventories, untracked warranty expirations, and missing configuration details across corporate subnets.

## 2. System Scope & Objectives

### 2.1 Objectives

- Provide a centralized, single-source-of-truth database for all network hardware assets (Routers, Switches, Firewalls, Access Points, Servers).
- Eliminate manual tracking errors by validating IP formats, MAC addresses, and subnet allocations.
- Enable rapid searching, filtering, and categorization of assets by location, status, vendor, and IP subnet.
- Deliver a responsive dashboard visualizing network asset distributions and critical alerts.

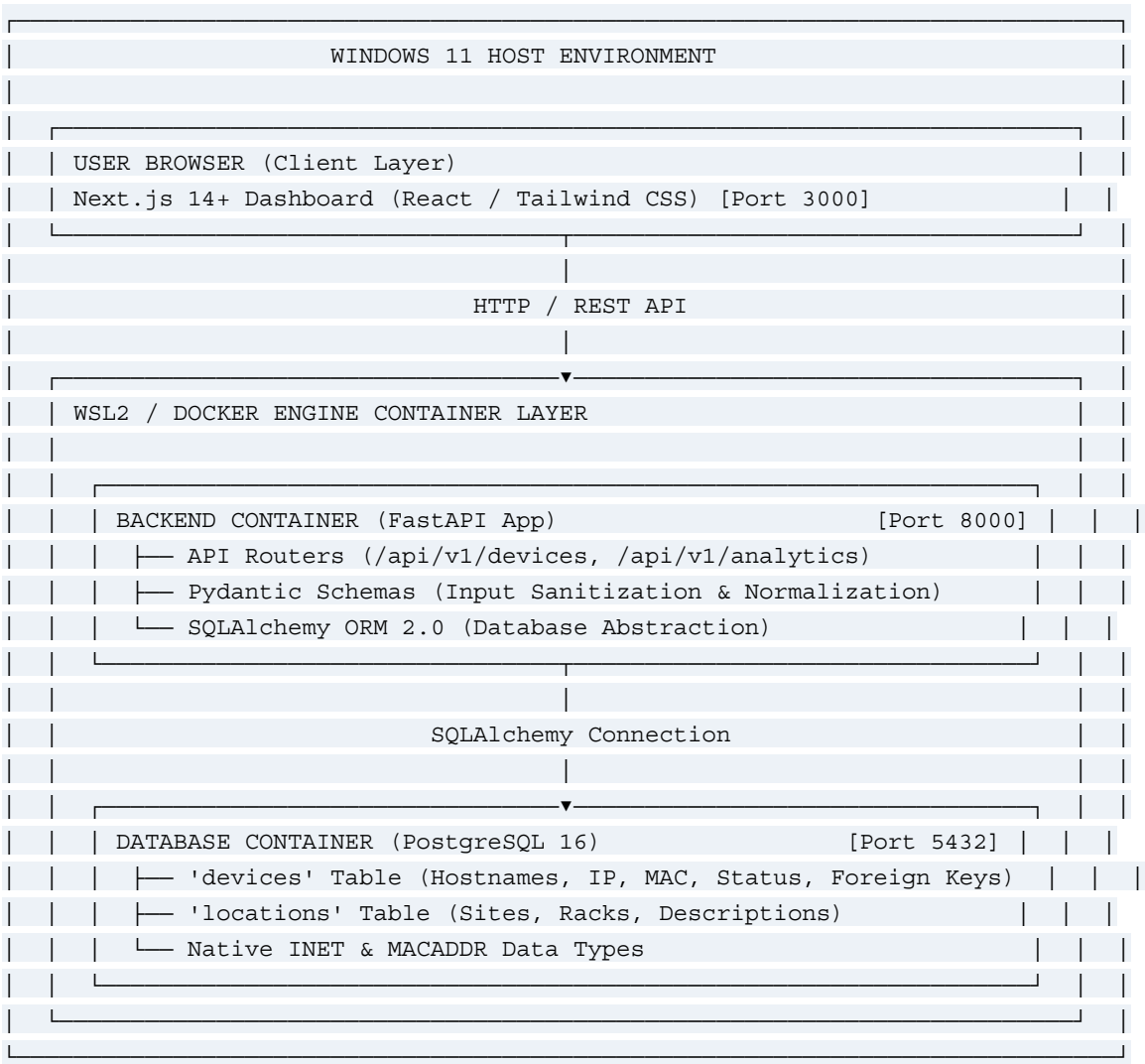
### 2.2 Target User Roles

| Role          | Permissions & Responsibilities                                                            |
|---------------|-------------------------------------------------------------------------------------------|
| Administrator | Full system access. Can create, read, update, and delete (CRUD) assets, manage locations, |

| Role                 | Permissions & Responsibilities                                               |
|----------------------|------------------------------------------------------------------------------|
|                      | tags, and export system reports.                                             |
| Read-Only Technician | Can search, view, and export device inventories without modification rights. |

### 3. Detailed System Architecture & Tech Stack

#### 3.1 Architectural Layout Diagram



### 3.2 Layer Specifications

| Layer             | Technology Selected               | Architecture & Rationale                                                                                                                                 |
|-------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Client Layer      | Next.js 14+ (React), Tailwind CSS | App Router architecture utilizing Server-Side Rendering (SSR), responsive layout components, and client-side form validation before API dispatch.        |
| Application Layer | FastAPI (Python 3.11+)            | Asynchronous REST API framework executing Pydantic v2 schemas for strict data typing, MAC address normalization, and OpenAPI interactive doc generation. |
| Persistence Layer | PostgreSQL 16 & SQLAlchemy 2.0    | Enterprise relational engine with native `INET` (IP address) and `MACADDR` (Hardware MAC) data types, orchestrated via type-safe SQLAlchemy ORM mapping. |
| Container Layer   | Docker Desktop & WSL2 (Ubuntu)    | Isolated containerized execution environment orchestrating PostgreSQL and FastAPI web services via `docker-compose.yml`.                                 |

## 4. Functional & API Requirements

---

### 4.1 REST API Endpoint Specification

| Method | Endpoint Route  | Description      | Status Code |
|--------|-----------------|------------------|-------------|
| GET    | /api/v1/devices | Paginated device | 200 OK      |

| Method | Endpoint Route            | Description                                                      | Status Code              |
|--------|---------------------------|------------------------------------------------------------------|--------------------------|
|        |                           | inventory list with multi-column sorting & search filters.       |                          |
| POST   | /api/v1/devices           | Register new network asset using Pydantic validated payload.     | 201 Created / 422 Error  |
| GET    | /api/v1/devices/{id}      | Retrieve detailed record for a specific asset by UUID.           | 200 OK / 404 Not Found   |
| PUT    | /api/v1/devices/{id}      | Update existing device specifications and location associations. | 200 OK / 400 Bad Request |
| DELETE | /api/v1/devices/{id}      | Remove asset record from inventory database.                     | 204 No Content           |
| GET    | /api/v1/analytics/summary | Fetch aggregate metrics for dashboard summary cards and charts.  | 200 OK                   |

## 5. Relational Database & ORM Model Architecture

---

### 5.1 PostgreSQL Table Schemas

Table: devices

|             |              |                                        |
|-------------|--------------|----------------------------------------|
| -----       |              |                                        |
| ---         |              |                                        |
| Column Name | Data Type    | Constraints                            |
| -----       |              |                                        |
| ---         |              |                                        |
| id          | UUID         | PRIMARY KEY, DEFAULT gen_random_uuid() |
| hostname    | VARCHAR(100) | UNIQUE, NOT NULL                       |

|                  |                   |                                                                                          |
|------------------|-------------------|------------------------------------------------------------------------------------------|
| ip_address       | INET              | UNIQUE, NOT NULL                                                                         |
| mac_address      | MACADDR           | UNIQUE, NULLABLE                                                                         |
| device_type      | VARCHAR(50)       | NOT NULL (e.g., 'Router', 'Switch', 'Firewall')                                          |
| vendor           | VARCHAR(50)       | NOT NULL (e.g., 'MikroTik', 'Cisco')                                                     |
| model            | VARCHAR(50)       | NOT NULL                                                                                 |
| serial_number    | VARCHAR(100)      | UNIQUE, NULLABLE                                                                         |
| firmware_version | VARCHAR(50)       | NULLABLE                                                                                 |
| status           | VARCHAR(20)       | DEFAULT 'Active' CHECK(status IN ('Active', 'Offline', 'Maintenance', 'Decommissioned')) |
| location_id      | UUID              | FOREIGN KEY references locations(id)                                                     |
| purchase_date    | DATE              | NULLABLE                                                                                 |
| warranty_expiry  | DATE              | NULLABLE                                                                                 |
| created_at       | TIMESTAMP WITH TZ | DEFAULT CURRENT_TIMESTAMP                                                                |
| updated_at       | TIMESTAMP WITH TZ | DEFAULT CURRENT_TIMESTAMP                                                                |

Table: locations

| Column Name   | Data Type    | Constraints                                     |
|---------------|--------------|-------------------------------------------------|
| id            | UUID         | PRIMARY KEY, DEFAULT gen_random_uuid()          |
| site_name     | VARCHAR(100) | NOT NULL (e.g., 'Headquarters', 'Branch Alpha') |
| building_rack | VARCHAR(50)  | NULLABLE (e.g., 'Building B - Rack 04')         |
| description   | TEXT         | NULLABLE                                        |

## 5.2 SQLAlchemy 2.0 Implementation Code Model

```
import uuid
from datetime import datetime
from sqlalchemy import String, Text, Date, DateTime, ForeignKey
from sqlalchemy.dialects.postgresql import UUID, INET, MACADDR
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Location(Base):
    __tablename__ = "locations"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    site_name: Mapped[str] = mapped_column(String(100), nullable=False)
    building_rack: Mapped[str | None] = mapped_column(String(50), nullable=True)
```

```

    description: Mapped[str | None] = mapped_column(Text, nullable=True)

    devices: Mapped[list["Device"]] = relationship("Device",
back_populates="location")

class Device(Base):
    __tablename__ = "devices"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    hostname: Mapped[str] = mapped_column(String(100), unique=True, nullable=False,
index=True)
    ip_address: Mapped[str] = mapped_column(INET, unique=True, nullable=False,
index=True)
    mac_address: Mapped[str | None] = mapped_column(MACADDR, unique=True,
nullable=True)
    device_type: Mapped[str] = mapped_column(String(50), nullable=False)
    vendor: Mapped[str] = mapped_column(String(50), nullable=False)
    model: Mapped[str] = mapped_column(String(50), nullable=False)
    serial_number: Mapped[str | None] = mapped_column(String(100), unique=True,
nullable=True)
    firmware_version: Mapped[str | None] = mapped_column(String(50), nullable=True)
    status: Mapped[str] = mapped_column(String(20), default="Active",
nullable=False)

    location_id: Mapped[uuid.UUID | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("locations.id"), nullable=True)
    purchase_date: Mapped[datetime | None] = mapped_column(Date, nullable=True)
    warranty_expiry: Mapped[datetime | None] = mapped_column(Date, nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=datetime.utcnow, onupdate=datetime.utcnow)

    location: Mapped[Location | None] = relationship("Location",
back_populates="devices")

```

## 6. Waterfall Phase Checklist & Status Sign-Off

---

| Phase Number   | Phase Name                       | Status           | Deliverables & Validation                                    |
|----------------|----------------------------------|------------------|--------------------------------------------------------------|
| <b>Phase 1</b> | Requirements & SRS Specification | <b>COMPLETED</b> | Functional, user, and domain scope signed off.               |
| <b>Phase 2</b> | System Design & Architecture     | <b>COMPLETED</b> | System Diagram, REST API Routes, and ORM Schema signed off.  |
| <b>Phase 3</b> | Implementation (Coding)          | Pending          | FastAPI Endpoints, Next.js Pages & UI Components.            |
| <b>Phase 4</b> | Testing & Validation             | Pending          | Pytest REST API unit tests, End-to-End integration checks.   |
| <b>Phase 5</b> | Deployment                       | Pending          | Docker Compose setup, GitHub release, local execution check. |